



LOVELY
PROFESSIONAL
UNIVERSITY

Project Name: Flash Card Game

Course Title: Summer Training

Name: Ritika

Registration Number: 12313655

**Name of the faculty: Dr. Preetjot
Kaur**

PROJECT REPORT

Introduction:

In today's fast-paced educational environment, interactive learning tools have become essential for effective knowledge acquisition and retention. Flashcard-based applications, in particular, have proven to be powerful aids in reinforcing concepts and promoting active recall. This report presents the development of a feature-rich flashcard game implemented in Java using the Swing graphical user interface (GUI) framework.

The **DSA Flashcard Game** is designed to provide users with an engaging platform to test and enhance their understanding of various topics through a series of question-and-answer flashcards. The application leverages core data structures such as ArrayList, HashSet, HashMap, Stack, and Queue to deliver functionalities like random question ordering, answer tracking, undo operations, and performance-based sorting. The intuitive user interface allows for seamless interaction, immediate feedback, and visual representation of progress, making learning both effective and enjoyable.

This project not only demonstrates the practical application of data structures and algorithms (DSA) in real-world software development but also showcases the integration of object-oriented programming principles and event-driven design. Through this report, we will explore the architecture, implementation details, and key features of the DSA Flashcard Game, highlighting its educational value and technical sophistication.

PURPOSE :

The primary purpose of the DSA Flashcard Game is to provide an interactive and user-friendly platform for students and learners to practice and reinforce their knowledge on various subjects using flashcards. By simulating a quiz-like environment, the application aims to:

- Enhance memory retention and recall through repeated exposure to questions.
- Encourage self-assessment by providing instant feedback and scoring.
- Demonstrate the practical application of fundamental data structures and algorithms in software development.

SCOPE :

The scope of the DSA Flashcard Game project encompasses the following features and functionalities:

- **Randomized Quiz Flow:** Flashcards are presented in random order to ensure varied practice sessions.
- **Answer Submission and Feedback:** Users can input their answers and receive immediate feedback on correctness.
- **Score Tracking:** The application maintains and displays the user's score, indicating performance.

- **Attempt Tracking:** The number of attempts for each flashcard is recorded, allowing for performance analysis and adaptive learning.
- **Undo Functionality:** Users can undo their last answer submission, promoting a forgiving learning environment.

- **Sorting by Difficulty:** Flashcards can be sorted based on the number of attempts, allowing users to focus on more challenging questions.
- **Visual Progress Representation:** At the end of the quiz, a graphical score wheel visually summarizes the user's performance.

- **User-Friendly Interface:** The application features an intuitive GUI built with Java Swing, ensuring accessibility and ease of use.

Overview:

The DSA Flashcard Game is a Java-based desktop application designed to help users practice and reinforce their knowledge through a series of interactive flashcards. The application presents questions in random order, tracks user performance, and provides immediate feedback. It incorporates essential data structures such as ArrayList,

HashSet, HashMap, Stack, and Queue to manage flashcards, track attempts, and implement features like undo and sorting by difficulty. The intuitive graphical user interface (GUI) built with Java Swing ensures a seamless and engaging user experience. The game concludes with a visual representation of the user's score, making learning both effective and enjoyable.

General Description :

The DSA Flashcard Game allows users to answer a set of predefined flashcard questions covering various topics. Each flashcard contains a question and its corresponding answer. The application shuffles the flashcards to present them in a random order, ensuring a unique experience for each session. Users enter their answers through the GUI, receive instant feedback, and can track their progress via a score display.

Key features include:

- **Immediate Feedback:** Users are notified instantly whether their answer is correct or incorrect.
- **Score Tracking:** The application maintains a running score and displays it throughout the quiz.
- **Attempt Tracking:** Each flashcard records the number of attempts, allowing users to identify challenging questions.

- **Undo Functionality:** Users can undo their last answer, which restores the previous question and adjusts the score and attempts accordingly.
- **Sorting by Attempts:** Users can sort flashcards based on difficulty (number of attempts) and restart the quiz to focus on harder questions.
- **Visual Score Summary:** At the end of the quiz, a graphical score wheel visually displays the user's performance.

General Constraints :

- **Platform:** The application is developed as a standalone Java Swing desktop application and requires the Java Runtime Environment (JRE) to execute.
- **Flashcard Set:** The current version uses a hardcoded set of flashcards; there is no built-in functionality for dynamically adding or importing new questions at runtime.
- **Single-User Mode:** The application is designed for single-user use and does not support multiple users or profiles.
- **Persistence:** User progress, scores, and attempts are not saved between sessions; all data is reset when the application is closed.

- **Input Validation:** The answer comparison is case-insensitive but does not handle spelling variations or partial matches.
- **Scalability:** While suitable for small to moderate sets of flashcards, performance may degrade with very large datasets due to in-memory data structure usage.
- **Accessibility:** The application relies on standard GUI components and does not include advanced accessibility features for users with disabilities.
- **External Dependencies:** The application uses only standard Java libraries and does not require any third-party dependencies.

GITHUB LINK:

<https://github.com/ritika1313/FlashCard-Game>

SnapShot Of Code:

```

1  import javax.swing.*;
2  import java.awt.*;
3  import java.awt.event.*;
4  import java.util.*;
5
6  public class DSAFlashcardGame extends JFrame {
7
8      // 1. ArrayList to store all flashcards
9      private ArrayList<Flashcard> flashcards = new ArrayList<>();
10
11     // 2. HashSet to track mastered flashcards (answered correctly at least once)
12     private HashSet<Flashcard> mastered = new HashSet<>();
13
14     // 3. HashMap to track number of attempts per flashcard
15     private HashMap<Flashcard, Integer> attempts = new HashMap<>();
16
17     // 4. Stack to keep history for undo functionality
18     private Stack<Flashcard> history = new Stack<>();
19
20     // 5. Queue to present flashcards in random order
21     private Queue<Flashcard> quizQueue = new LinkedList<>();
22
23     // Current flashcard being shown
24     private Flashcard currentCard;
25
26     // Score and question count
27     private int score = 0;
28     private int totalQuestions = 0;
29
30     // GUI Components
31     private JLabel questionLabel;
32     private JTextField answerField;
33     private JButton submitButton, nextButton, undoButton, sortButton;
34     private JLabel feedbackLabel, scoreLabel;
35
36     public DSAFlashcardGame() {
37         setTitle(title: "DSA Flashcard Game");

```

```

36 public DSAFlashcardGame() {
37     setTitle(title:"DSA Flashcard Game");
38     setSize(width:600, height:350);
39     setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
40     setLocationRelativeTo(c:null);
41
42     // Initialize flashcards
43     initializeFlashcards();
44
45     // Initialize quiz queue with shuffled flashcards
46     startQuiz();
47
48     // Setup GUI layout
49     setupGUI();
50
51     // Show first question
52     showNextQuestion();
53
54     setVisible(b:true);
55 }
56
57 // Initialize flashcards with 11+ sample questions
58 private void initializeFlashcards() {
59     flashcards.add(new Flashcard(question:"What is the capital of France?", answer:"Paris"));
60     flashcards.add(new Flashcard(question:"2 + 2 = ?", answer:"4"));
61     flashcards.add(new Flashcard(question:"What color do you get by mixing red and white?", answer:"Pink"));
62     flashcards.add(new Flashcard(question:"Who wrote 'Romeo and Juliet'?", answer:"Shakespeare"));
63     flashcards.add(new Flashcard(question:"What is the boiling point of water (in Celsius)?", answer:"100"));
64     flashcards.add(new Flashcard(question:"What is the largest planet in our solar system?", answer:"Jupiter"));
65     flashcards.add(new Flashcard(question:"What year did World War II end?", answer:"1945"));
66     flashcards.add(new Flashcard(question:"What is the chemical symbol for Gold?", answer:"Au"));
67     flashcards.add(new Flashcard(question:"Who painted the Mona Lisa?", answer:"Leonardo da Vinci"));
68     flashcards.add(new Flashcard(question:"What is the square root of 64?", answer:"8"));
69     flashcards.add(new Flashcard(question:"What language is primarily spoken in Brazil?", answer:"Portuguese"));
70     totalQuestions = flashcards.size();
71 }

```

```
73 // Shuffle flashcards and load into queue
74 private void startQuiz() {
75     Collections.shuffle(flashcards); // Shuffle for random order
76     quizQueue.clear();
77     quizQueue.addAll(flashcards);
78     score = 0;
79     mastered.clear();
80     attempts.clear();
81     history.clear();
82 }
83
84 // Setup all GUI components and layout
85 private void setupGUI() {
86     setLayout(new BorderLayout(hgap:10, vgap:10));
87
88     // Question display at top
89     questionLabel = new JLabel(text:"", SwingConstants.CENTER);
90     questionLabel.setFont(new Font(name:"Arial", Font.BOLD, size:20));
91     add(questionLabel, BorderLayout.NORTH);
92
93     // Center panel for answer input and buttons
94     JPanel centerPanel = new JPanel(new FlowLayout());
95     centerPanel.add(new JLabel(text:"Your Answer: "));
96     answerField = new JTextField(columns:20);
97     centerPanel.add(answerField);
98
99     submitButton = new JButton(text:"Submit");
100    centerPanel.add(submitButton);
101
102    nextButton = new JButton(text:"Next");
103    nextButton.setEnabled(b:false);
104    centerPanel.add(nextButton);
105
106    undoButton = new JButton(text:"Undo");
107    undoButton.setEnabled(b:false);
108    centerPanel.add(undoButton);
```

```

// Update attempt count for this flashcard
attempts.put(currentCard, attempts.getDefault(currentCard, defaultValue:0) + 1);

// Check answer correctness
if (userAnswer.equalsIgnoreCase(currentCard.getAnswer())) {
    feedbackLabel.setText(text:"Correct! Well done.");
    score++;
    mastered.add(currentCard); // Mark as mastered
} else {
    feedbackLabel.setText("Incorrect! The correct answer is: " + currentCard.getAnswer());
}

// Push current card to history for undo
history.push(currentCard);

// Update buttons and score
submitButton.setEnabled(b:false);
nextButton.setEnabled(b:true);
undoButton.setEnabled(b:true);
updateScoreLabel();
}

// Undo the last answer submission
private void undoLastAnswer() {
    if (history.isEmpty()) {
        JOptionPane.showMessageDialog(this, message:"No actions to undo.", title:"Info", JOptionPane.INFORMATION_MESSAGE);
        return;
    }

    Flashcard lastCard = history.pop();
    // Adjust score if last answer was correct
    if (mastered.contains(lastCard)) {
        score--;
        mastered.remove(lastCard);
    }
}

```

```

Flashcard lastCard = history.pop();
// Adjust score if last answer was correct
if (mastered.contains(lastCard)) {
    score--;
    mastered.remove(lastCard);
}

// Decrement attempts count for last card
int count = attempts.getDefault(lastCard, defaultValue:1);
if (count <= 1) {
    attempts.remove(lastCard);
} else {
    attempts.put(lastCard, count - 1);
}

// Put the last card back at the front of the queue for retry
LinkedList<Flashcard> tempList = new LinkedList<>(quizQueue);
tempList.addFirst(lastCard);
quizQueue = tempList;

// Show undone card again
currentCard = lastCard;
questionLabel.setText("<html><center>" + currentCard.getQuestion() + "</center></html>");
answerField.setText(t:"");
feedbackLabel.setText(text:"Last answer undone. Please try again.");
submitButton.setEnabled(b:true);
nextButton.setEnabled(b:false);
undoButton.setEnabled(b:false);
answerField.requestFocus();

updateScoreLabel();
}

```

```
// Custom JPanel to draw the green score wheel at the end
class ScoreWheelPanel extends JPanel {
    private int score;
    private int total;

    public ScoreWheelPanel(int score, int total) {
        this.score = score;
        this.total = total;
        setPreferredSize(new Dimension(width:250, height:250));
        setBackground(Color.WHITE);
    }

    @Override
    protected void paintComponent(Graphics g) {
        super.paintComponent(g);
        // Draw green circle
        g.setColor(new Color(r:34, g:177, b:76)); // Green color
        g.fillOval(x:25, y:25, width:200, height:200);

        // Draw score text
        g.setColor(Color.WHITE);
        g.setFont(new Font(name:"Arial", Font.BOLD, size:40));
        String text = score + " / " + total;
        FontMetrics fm = g.getFontMetrics();
        int x = getWidth()/2 - fm.stringWidth(text)/2;
        int y = getHeight()/2 + fm.getAscent()/2;
        g.drawString(text, x, y);

        // Draw "Game Over" text above the circle
        g.setFont(new Font(name:"Arial", Font.BOLD, size:24));
        String gameOver = "Game Over!";
        int x2 = getWidth()/2 - g.getFontMetrics().stringWidth(gameOver)/2;
        g.drawString(gameOver, x2, y:50);
    }
}
```

GUI Snapshot:



What year did World War II end?

Your Answer:

Submit

Next

Undo

Sort by Attempts

Score: 0 / 1